

PROPOSAL · FOR DISCUSSION

Code-Driven Motion Graphics

A practical way to produce lower thirds, bumpers, timelines, scripture cards, maps and short animated films. Every frame is written in code, AI assistants do the building, and people keep every decision.

94

finished clips

23

visual styles

12

languages from one template

2

narrated explainers

1

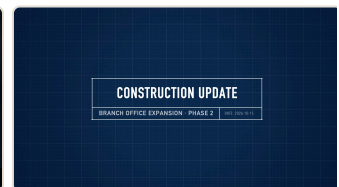
laptop · overnight

Prepared by Cole Jarrett · September 28, 2026

1. Summary

Each graphic in this library is a small program. The program answers one question: what does the picture look like at time t ? Because every frame depends only on time and on the text supplied, frames can be rendered on any computer, in any order and in parallel, and they come out identical every time. An AI assistant writes and revises the programs from a short brief. A second AI model critiques the rendered frames. A person approves the look, the wording and the final cut.

Why this matters for a translation-heavy organisation: the text is data. A spreadsheet of names, dates or verse references becomes finished clips in every language, and scripts like Arabic mirror automatically. Changing one name means re-rendering one clip in seconds, with no project files to reopen.



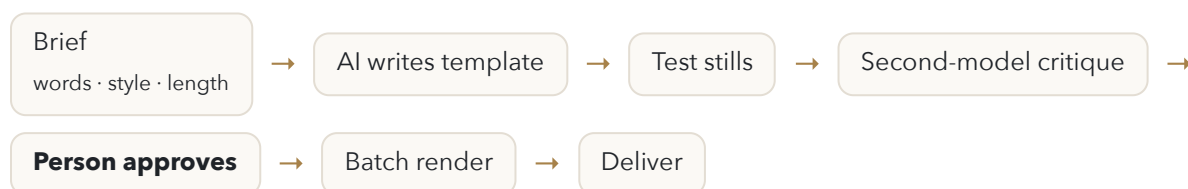


2. What was built for this meeting

In a single overnight session on one Apple M1 laptop (8 GB):

- **94 clips in 23 styles:** Archival, Blueprint, Botanical, Chalkboard, Editorial Serif, Engraving, Infographic, Isometric, Kinetic Type, Letterpress, Soft Light, Bible Maps, Clean Minimal, Ancient Mosaic, Night Sky, Paper Cut, Riso Print, Sketchbook, Ink Wash, Swiss Grid, Topographic, Watercolor, and the Gilded Relief film engine.
- **12-language demonstration:** the same lower third rendered from a table of names in English, Spanish, French, Portuguese, German, Russian, Japanese, Korean, Chinese, Arabic (right-to-left), Hindi and Swahili.
- **Two narrated explainers and a style reel:** “How it works” and “The cost picture” were built with the same toolkit and narrated by an on-device speech model.
- **Case study:** a 3-minute animated film of Daniel 2:31–45 in a gilded-relief style, fully narrated with sound design.
- **Original music and sound:** every bumper carries a short, calm score synthesized in code, so there is nothing to license.

3. How it works



PIECE	WHAT IT IS	WHY IT MATTERS
Template	~100–300 lines of JavaScript drawing on a canvas, with all text and colours in parameters	Readable, versionable and reviewable; one template → many clips
Renderer	A standard browser (Chrome) running headless, driven frame by frame	No special software; runs on any Mac or PC
Encoder	ffmpeg (open source)	ProRes 4444 with alpha for editors, and H.264 for review

Narration	Kokoro, an open speech model running offline on the laptop	Scratch or temp narration with no data leaving the machine
Music	Synthesized in code: piano, pads, bells, plucks	Original and license-free; calm by design
AI	Claude (building and fixing), Gemini (independent critique and extra packs)	Two models check each other; a person decides

No generated imagery. AI is used only to write drawing code. Every shape is plain geometry and every word is text we supply, so there is no synthetic photography, no likeness of real people, and nothing the model “imagined”. Each clip can be inspected line by line.

4. Measured time and cost

ITEM	MEASURED
Daniel 2 film: human input	8 short chat messages across one afternoon (direction, translation choice, verse range, text, voice choice)
Daniel 2 film: AI work	Planned 6 scenes, wrote 1,427 lines of scene code and 100+ sound cues, and checked frames for defects
Daniel 2 film: final render	5,640 frames at 1080p, 22 minutes on the laptop in the background
This library: render compute	76 minutes total for 29,175 frames (median 131 ms per frame, 2-3 workers)
One lower third	≈ 21 s to render (210 frames), including the ProRes and H.264 encodes
Hardware	One Apple M1 laptop, 8 GB RAM. No render farm and no GPU server.
Software licences	None required for the pipeline: Chrome, ffmpeg and the speech model are free
AI cost	Covered by standard AI subscriptions (Claude and Gemini seats); no per-clip fees

Comparisons with traditional workflows in the explainer are illustrative estimates. A pilot (section 7) would measure them properly against the team’s real jobs.

5. Where it fits and where it does not

Strong fits

- Recurring graphics: lower thirds, location tags, scripture callouts, agendas, countdowns, program bumpers
- Anything that multiplies by language: re-render the same design from a translated table
- Explanatory graphics: timelines, maps with routes, simple diagrams and data
- Consistency: a design system encoded once and applied the same way everywhere

Honest limitations

- This is illustration and typography, not photorealism or character animation.
- Production would use the organisation’s own fonts and design standards. The demo uses system fonts.
- AI-written code still needs an experienced eye. The critique loop catches a lot, but not everything.

- Synthesized voice and music suit scratch tracks and internal pieces. Published work would keep human narration and approved music.
- Quality is set by review. The speed only helps if approval stays with people.

6. How it would slot into existing work

- **Editors** receive ProRes 4444 files with transparency, which drop straight into Premiere, Resolve or Final Cut, as well as H.264 previews.
- **Templates** live in a shared repository with version history, and a change to a template is reviewed like a document edit.
- **Translation teams** supply text tables. Clips come back per language, with fonts chosen per script.
- **Rendering** runs on a spare Mac or PC, or on several machines at once, since frames are independent.

7. Suggested next step: a four-week pilot

WEEK	GOAL
1	Choose three recurring graphics currently made by hand. Encode the team's fonts, colours and spacing as a template "design system".
2	Build the three templates with AI assistance, and review with the design team.
3	Render them in 5-10 languages from real translated tables, and hand them to editors.
4	Compare hours and revisions against the current process, then decide whether to continue.

Appendix: files in the demo folder

Library.html	The catalog: every clip, with hover preview, playback and the template code
explainers/	The three narrated explainers
out/<style>/	Rendered clips: .mp4 previews, .mov ProRes with alpha, and .jpg posters
templates/	The source of every graphic, one small file each
engine/	The runtime (kit.js), the renderer, batch build, music and scoring
TEMPLATES.md	The authoring guide and house rules given to every AI worker

All names in the demo are fictional. Scripture appears as references, apart from the Daniel 2 text supplied for the case study. No logos or branding of any organisation are used.